# Cereal 2 — Environment Setup

Oct 26, 2026 · Anas Alsirafy

The maize notebooks need a GPU with enough memory for ConvNeXt-Base, TensorFlow/Keras, and the `iterative-stratification` package. The full specialist run is long, so it is built to survive interruptions.

## Hardware

| Machine | Notes |
|---|---|
| Workstation, NVIDIA RTX 5090 (WSL2 on Windows) | Main training machine for this group's work |
| Kaggle notebook (GPU model to confirm — written as P100) | Alternative when the workstation is busy; sessions time out |

Which of the two ran the final maize numbers is not recorded; ask Anas.

## Software

| Package | Note |
|---|---|
| Python 3.x, TensorFlow / Keras | Exact versions to confirm from the notebooks' first cell |
| scikit-learn | StratifiedKFold for the binary specialists, metrics |
| iterative-stratification | `pip install iterative-stratification`, then `from iterstrat.ml_stratifiers import MultilabelStratifiedKFold`. The import name differs from the package name. |
| opencv-python | CLAHE pre-processing |
| albumentations | Augmentation |
| numpy, pandas | Labels and OOF arrays |

## Setup steps

1. Copy the project folder from Google Drive (link to be added).
2. On Windows, work inside WSL2; native Windows TensorFlow has no GPU support after 2.10.
3. `pip install tensorflow scikit-learn iterative-stratification opencv-python albumentations pandas`.
4. Check the GPU: `python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"`.
5. Download the maize images (Kaggle mirror of Wiesner-Hanks 2018) and place `Database.csv` next to them; see the Data doc.
6. Regenerate the notebooks from `build_maize_nb.py` / `build_specialist_nb.py` if you change them — edit the builder, not the notebook.
7. Smoke test: run fold 1 of the monolithic notebook.

## Surviving long runs

The specialist notebook trains 5 models × 5 folds = 25 runs, five times as many training runs as the monolithic model.

- `specialist_checkpoint.json` records every finished (specialist, fold) pair with its metrics. On re-run, finished pairs are skipped.
- OOF probabilities are saved after every fold: `oof_binary_{disease}.npy` for S1–S4 and `oof_final.npy` for S5, so evaluation cells work after a partial run.
- `ModelCheckpoint` saves during each phase, so a crash costs one phase, not one fold.
- Save models as `.keras`, not `.h5`.